pyhula — Python Interface Reference (V3)

English translation of `Hula Python接口说明 V3_20250724.pdf` (source document is a rendering of `readmewhl.md`, dated 2025-07-18).

Chinese dependency package for hula Python programming. A Python pack used by hula.

Python version: 3.6.7

Installing

Run either of the following commands in a terminal to install pyhula:

```
pip install pyhula
```

```
pip install pyhula-1.0.4-cp36-cp36m-win_amd64.whl
```

Checking the version

Run `pip list` in the console to check it, or from within a program:

```
import pyhula
ver = pyhula.get_version()
print(ver)
```

Usage

Use the code below to obtain a `UserApi` instance; the interfaces it provides can then be used to control the hula drone. For the interface documentation, see `doc/html/中文/index.html` (English: `doc/html/English/index.html`).

```
import pyhula
api = pyhula.UserApi()
if not api.connect():
    print("connect error")
else:
    print('connection to station by wifi')

api.single_fly_takeoff()    # take off
api.single_fly_touchdown()  # land
```

Interface reference

Throughout this document, the `led` parameter has the same meaning:

> **led** — optional, defaults to `0`. Format `{'r':0,'g':0,'b':0,'mode':1}`, where `r`, `g`, `b` are the colour values and `mode` is one of: `1` steady on, `2` off, `4` RGB three-colour cycle, `16` seven-colour (rainbow), `32` flashing, `64` breathing.

Connect to the drone

```
connect(server_ip)
```

Description: Connects to the drone.

Parameters:

- *optional* `server_ip` — the drone's IPv4 address. If omitted, it is discovered automatically.

Returns: `True` on success, `False` on failure.

Example:

```
api.connect('192.168.1.118')
api.connect()
```

Take off

```
single_fly_takeoff(led)
```

Description: Real-time control — makes the drone take off.

Parameters:

- `led` — optional, defaults to `0`. See the LED note above.

Example:

```
api.single_fly_takeoff()
api.single_fly_takeoff({'r':16,'g':15,'b':100,'mode':1})
```

Land

```
single_fly_touchdown(led)
```

Description: Real-time control — makes the drone land.

Parameters:

- `led` — optional, defaults to `0`. See the LED note above.

Example:

```
api.single_fly_touchdown()
api.single_fly_touchdown({'r':16,'g':15,'b':100,'mode':1})
```

Hover

```
single_fly_hover_flight(time, led)
```

Description: The drone hovers in place.

Parameters:

- `time` — hover duration (seconds).

Example:

```
api.single_fly_hover_flight(10)
api.single_fly_hover_flight(10,{'r':16,'g':15,'b':100,'mode':1})
```

Fly forward

```
single_fly_forward(distance, speed, led)
```

Description: Real-time control — flies the drone forward.

Parameters:

- `distance` — flight distance (cm).
- `speed` — optional, defaults to `100`. Speed, 0–100 cm/s.

- `led` — optional, defaults to `0`. See the LED note above.

Example:

```
api.single_fly_forward(100)
api.single_fly_forward(100,100,{'r':16,'g':15,'b':100,'mode':1})
```

Fly backward

```
single_fly_back(distance, speed, led)
```

Description: Real-time control — flies the drone backward.

Parameters:

- `distance` — flight distance (cm).
- `speed` — optional, defaults to `100`. Speed, 0–100 cm/s.
- `led` — optional, defaults to `0`. See the LED note above.

Example:

```
api.single_fly_back(100)
api.single_fly_back(100,100,{'r':16,'g':15,'b':100,'mode':1})
```

Fly left

```
single_fly_left(distance, speed, led)
```

Description: Real-time control — flies the drone to the left.

Parameters:

- `distance` — flight distance (cm).
- `speed` — optional, defaults to `100`. Speed, 0–100 cm/s.
- `led` — optional, defaults to `0`. See the LED note above.

Example:

```
api.single_fly_left(100)
api.single_fly_left(100,100,{'r':16,'g':15,'b':100,'mode':1})
```

Fly right

```
single_fly_right(distance, speed, led)
```

Description: Real-time control — flies the drone to the right.

Parameters:

- `distance` — flight distance (cm).
- `speed` — optional, defaults to `100`. Speed, 0–100 cm/s.
- `led` — optional, defaults to `0`. See the LED note above.

Example:

```
api.single_fly_right(100)
api.single_fly_right(100,100,{'r':16,'g':15,'b':100,'mode':1})
```

Fly up

```
single_fly_up(distance, speed, led)
```

Description: Real-time control — flies the drone upward.

Parameters:

- `height` — flight height (cm). *(The original names the parameter `height` here even though the signature reads `distance`.)*
- `speed` — optional, defaults to `100`. Speed, 0–100 cm/s.
- `led` — optional, defaults to `0`. See the LED note above.

Example:

```
api.single_fly_up(100)
api.single_fly_up(100,100,{'r':16,'g':15,'b':100,'mode':1})
```

Fly down

```
single_fly_down(distance, speed, led)
```

Description: Real-time control — flies the drone downward.

Parameters:

- `height` — flight height (cm). *(The original names the parameter `height` here even though the signature reads `distance`.)*
- `speed` — optional, defaults to `100`. Speed, 0–100 cm/s.
- `led` — optional, defaults to `0`. See the LED note above.

Example:

```
api.single_fly_down(100)
api.single_fly_down(100,100,{'r':16,'g':15,'b':100,'mode':1})
```

Rotate left

```
single_fly_turnleft(angle, led)
```

Description: Real-time control — turns the drone to the left.

Parameters:

- `angle` — rotation angle (degrees).
- `led` — optional, defaults to `0`. See the LED note above.

Example:

```
api.single_fly_turnleft(90)
api.single_fly_turnleft(90,{'r':16,'g':15,'b':100,'mode':1})
```

Rotate right

```
single_fly_turnright(angle, led)
```

Description: Real-time control — turns the drone to the right.

Parameters:

- `angle` — rotation angle (degrees).
- `led` — optional, defaults to `0`. See the LED note above.

Example:

```
api.single_fly_turnright(90)
api.single_fly_turnright(90,{'r':16,'g':15,'b':100,'mode':1})
```

Bounce

```
single_fly_bounce(frequency, height, led)
```

Description: Real-time control — makes the drone bounce.

Parameters:

- `frequency` — number of bounces.
- `height` — bounce distance (cm).
- `led` — optional, defaults to `0`. See the LED note above.

Example:

```
api.single_fly_bounce(3, 50)
api.single_fly_bounce(3, 50,{'r':16,'g':15,'b':100,'mode':1})
```

Straight-line flight

```
single_fly_straight_flight(x, y, z, speed, led)
```

Description: Flies in a straight line to (x, y, z).

Parameters:

- `x` — x coordinate (cm).
- `y` — y coordinate (cm).
- `z` — z coordinate (cm).
- `speed` — optional, defaults to `100`. Speed, 0–100 cm/s.
- `led` — optional, defaults to `0`. See the LED note above.

Example:

```
api.single_fly_straight_flight(100, 100, 100)
api.single_fly_straight_flight(100, 100, 100,50,
                               {'r':16,'g':15,'b':100,'mode':1})
```

Circular flight

```
single_fly_radius_around(radius, led)
```

Description: Flies a circle of the given radius.

Parameters:

- `radius` — orbit radius (cm; positive = counter-clockwise, negative = clockwise).
- `led` — optional, defaults to `0`. See the LED note above.

Example:

```
api.single_fly_radius_around(100)
api.single_fly_radius_around(100,{'r':16,'g':15,'b':100,'mode':1})
```

Spin in place

```
single_fly_autogyration360(num, led)
```

Description: Spins the drone in place, clockwise or counter-clockwise, for a given number of turns.

Parameters:

- `num` — number of turns (positive = counter-clockwise, negative = clockwise).

- `led` — optional, defaults to `0`. See the LED note above.

Example:

```
api.single_fly_autogyration360(2)
api.single_fly_autogyration360(2,{'r':16,'g':15,'b':100,'mode':1})
```

Flip

```
single_fly_somersault(direction)
```

Description: The drone flips in place — forward, backward, left or right.

Parameters:

```
DIRECTION_FORWARD = 0,  /* forward. | */
DIRECTION_BACK    = 1,  /* back.    | */
DIRECTION_LEFT    = 2,  /* left.    | */
DIRECTION_RIGHT   = 3,  /* right.   | */
```

- `led` — optional, defaults to `0`. See the LED note above.

Example:

```
api.single_fly_somersault(0)
api.single_fly_somersault(0,{'r':16,'g':15,'b':100,'mode':1})
```

Curved flight

```
single_fly_curvilinearFlight(x, y, z, direction, speed, led)
```

Description: Curved flight to (x, y, z).

Parameters:

- `x` — x-axis coordinate (cm) (left/right of the airframe; right is positive).
- `y` — y-axis coordinate (cm) (fore/aft of the airframe; forward is positive).
- `z` — z-axis coordinate (cm) (up/down of the airframe; up is positive).
- `direction` — `True` = counter-clockwise, `False` = clockwise. Defaults to `True`.
- `speed` — optional, defaults to `100`. Speed, 0–100 cm/s.
- `led` — optional, defaults to `0`. See the LED note above.

Example:

```
api.single_fly_curvilinearFlight(100, 100, 0, True, 50)
api.single_fly_curvilinearFlight(100, 100, 0, False, 50,
                                 {'r':16,'g':15,'b':100,'mode':1})
```

Enable obstacle avoidance

```
single_fly_barrier_aircraft(mode)
```

Description: Turns obstacle avoidance on.

Parameters:

- `mode` — `True` = on, `False` = off.

Example:

```
api.single_fly_barrier_aircraft(True)
```

Line-following detection

```
single_fly_Line_walking(fun_id, dist, way_color)
```

Description: Line-following detection.

Parameters:

- `fun_id = 0` — `0`: follow the line forward, ignoring junctions.
- `dist` — distance, in cm.
- `way_color` — colour of the line to follow: `0` = black, `255` = white.

Returns:

- `result` — result of the command: `0` = failure, `1` = success, `2` = success, junction reached.

Example:

```
api.single_fly_Line_walking(0, 100, 0)
```

Tag recognition

```
single_fly_AiIdentifies(mode)
```

Description: Recognises a tag.

Parameters:

- `mode` — `0`–`9` recognise the number tags 0–9; `10` left arrow; `11` right arrow;

`12` up arrow; `13` down arrow; `20` end the task; `65`–`90` the uppercase letters A–Z. Once triggered, recognition runs for 300 ms; if it succeeds it ends immediately.

Returns:

- `x` — X coordinate of the tag card relative to the drone.
- `y` — Y coordinate of the tag card relative to the drone.
- `z` — Z coordinate of the tag card relative to the drone.
- `angle` — angle of the tag card relative to the drone.
- `result` — `False` = recognition failed, `True` = recognition succeeded.

Example:

```
api.single_fly_AiIdentifies(1)
```

Optical-flow QR code alignment

```
single_fly_Optical_flow_alignment(qr_id, qr_size, angle = 0)
```

Description: Aligns to a QR code using the optical-flow (downward) camera.

Parameters:

- `qr_id` — target QR code ID, `[0-9]`.
- `qr_size` — physical size of the QR code, range `[6, 30]`, default `20`, in cm.
- `angle` — align to the QR code and then rotate by n degrees.

Returns:

- `result` — `False` = recognition failed, `True` = recognition succeeded.

Example:

```
api.single_fly_Optical_flow_alignment(1, 20, 0)
```

Optical-flow QR code recognition

```
single_fly_Optical_flow_recognition(qr_id, qr_size)
```

Description: Recognises a QR code using the optical-flow (downward) camera.

Parameters:

- `qr_id` — target QR code ID, `[0-9]`.
- `qr_size` — physical size of the QR code, range `[6, 30]`, default `20`, in cm.

Returns:

```
{
    result;  // False = recognition failed, True = recognition succeeded
    x;       // distance between the drone and the QR code
    y;       // distance between the drone and the QR code
    z;       // distance between the drone and the QR code
    yaw;     // angle between the drone and the QR code
    qr_id;   // ID of the recognised QR code
}
```

Example:

```
api.single_fly_recognition_Qrcode(0, 1)
```

(The example in the original names a different function than the heading — reproduced as printed.)

Front-camera QR code alignment

```
single_fly_Proactive_alignment(qr_id)
```

Description: Aligns to a QR code using the front camera.

Parameters:

- `qr_id` — target QR code ID, `[0-9]`.

Returns:

- `result` — `False` = recognition failed, `True` = recognition succeeded.

Example:

```
api.single_fly_Proactive_alignment(1)
```

Front-camera QR code recognition

```
single_fly_Anticipatory_recognition(qr_id)
```

Description: Recognises a QR code using the front camera.

Parameters:

- `qr_id` — target QR code ID, `[0-9]`.

Returns:

```
{
    result;  // False = recognition failed, True = recognition succeeded
    x;       // distance between the drone and the QR code
    y;       // distance between the drone and the QR code
    z;       // distance between the drone and the QR code
    yaw;     // angle between the drone and the QR code
    qr_id;   // ID of the recognised QR code
}
```

Example:

```
api.single_fly_Anticipatory_recognition(1)
```

Track a QR code

```
single_fly_track_Qrcode(qr_id, time)
```

Description: Tracks QR code `[0-9]` for `[time]` seconds.

Parameters:

- `qr_id` — QR code ID.
- `time` — tracking duration.

Returns:

- `result` — `0` = success, `1` = failure.

Example:

```
api.single_fly_track_Qrcode(1, 10)
```

Colour recognition — read the colour of one video frame

```
single_fly_getColor()
```

Description: Colour recognition; reads the colour from one frame of the current video stream.

Parameters:

- `Mode` — `1` = start, runs one frame.

Returns:

- `r, g, b` — colour values.
- `state` — `0` = failure, `1` = success.

Example:

```
ret = api.single_fly_getColor()  # returns: r,g,b colour values, state: 0 failure / 1 success
```

Set the LED colour and mode (does not block the main thread)

```
single_fly_lamplight(r, g, b, time, mode)
```

Description: Sets the LED colour and mode.

Parameters:

- `r, g, b` — colour values.
- `time` — light duration, in seconds.
- `mode` — `1` steady on, `2` off, `4` RGB three-colour cycle, `16` seven-colour (rainbow), `32` flashing, `64` breathing.

Returns: `True` = executed successfully, `False` = execution failed.

Example:

```
api.single_fly_lamplight(255, 0, 0, 1, 1)  # set LED colour and mode
```

Fire the laser

```
plane_fly_generating(type, data, reserve)
```

Description: Fires the laser.

Parameters:

- `type = 0` — laser: `0` single shot, `1` burst, `2` enable laser reception,

`3` disable laser reception, `4` continuous burst with unlimited ammo, `5` stop firing.

- `data = 10` — laser burst rate, shots per second, range 1–14.
- `reserve = 100` — ammo count, range 1–255.

Example:

```
api.plane_fly_generating(0, 10, 100)  # single shot
api.plane_fly_generating(2, 10, 100)  # enable laser reception
```

Laser receiver hit

```
plane_fly_laser_receiving()
```

Description: Reports whether the laser receiver has been hit.

Returns: `True` = hit, `False` = not hit.

Example:

```
api.plane_fly_laser_receiving()
```

QR code positioning switch

```
Plane_cmd_switch_QR(type)
```

Description: Turns QR code positioning on or off.

Parameters:

- `type` — `0` = enable QR code positioning, `1` = disable QR code positioning.

Example:

```
api.Plane_cmd_switch_QR(0)
```

Take a photo

```
Plane_fly_take_photo()
```

Description: Takes a photo. The video stream must be enabled before calling this.

Example:

```
api.Plane_fly_take_photo()  # take a photo
```

Record video

```
Plane_cmd_switch_video(type)
```

Description: Starts recording video.

Parameters:

- `type` — video recording: `0` = start, `1` = stop.

Example:

```
api.Plane_cmd_switch_video(0)  # start recording
```

Enable the video stream

```
Plane_cmd_swith_rtp(type)
```

Description: Enables the video stream.

Parameters:

- `type` — `0` = enable, `1` = disable.

Example:

```
api.Plane_cmd_swith_rtp(0)  # enable the video stream
```

Open the video stream window

```
single_fly_flip_rtp()
```

Description: Opens the video stream window. The video stream must be enabled before calling this.

Parameters: none.

Example:

```
api.single_fly_flip_rtp()  # open the video stream window
```

Set the main camera pitch angle

```
Plane_cmd_camera_angle(type, data)
```

Description: Sets the pitch angle of the main camera.

Parameters:

- `type = 0` — direction of travel: `0` up, `1` down (absolute), `2` and `3` algorithm-controlled, `4` calibrate, `5` block up, `6` block down (relative).
- `data = 30` — angle of travel: 0–90.

Example:

```
api.Plane_cmd_camera_angle(0, 30)  # set the main camera pitch angle
```

Spin the propellers at low speed

```
plane_fly_arm()
```

Description: Arms the motors.

Parameters: none.

Example:

```
api.plane_fly_arm()  # spin the propellers at low speed
```

Stop spinning the propellers at low speed

```
plane_fly_disarm()
```

Description: Disarms the motors.

Parameters: none.

Example:

```
api.plane_fly_disarm()  # stop spinning the propellers at low speed
```

Get obstacle-avoidance information

```
Plane_getBarrier()
```

Description: Gets obstacle-avoidance information.

Parameters: none.

Returns: a dictionary giving the obstacle state in each direction — `True` = obstacle present, `False` = no obstacle.

```
{
    'forward': True
    'back':    True,
    'left':    True,
    'right':   True,
}
```

Example:

```
ret = api.Plane_getBarrier()  # get obstacle-avoidance information
```

Get the drone's battery percentage

```
get_battery()
```

Description: Gets the drone's battery percentage.

Returns: integer — battery percentage.

Example:

```
ret = api.get_battery()  # get the drone's battery percentage
```

Get the drone's coordinates (x, y, z)

```
get_coordinate()
```

Description: Gets the drone's coordinates `[x, y, z]`.

Parameters: none.

Returns: `[x, y, z]`

Example:

```
ret = api.get_coordinate()  # get the drone's coordinates [x, y, z]
```

Get the drone's attitude angles

```
get_yaw()
```

Description: Gets the drone's angles (in degrees).

Returns: integers — `[yaw, pitch, roll]`

Example:

```
ret = api.get_yaw()
```

Get the drone's body-frame velocity (X, Y, Z)

```
get_plane_speed()
```

Description: Gets the drone's body-frame velocity (X-axis, Y-axis, Z-axis).

Returns: integers — `[X, Y, Z]`

Example:

```
ret = api.get_plane_speed()
```

Get the drone's ToF height

```
get_plane_distance()
```

Description: Gets the drone's time-of-flight height.

Returns: integer — the drone's ToF height.

Example:

```
ret = api.get_plane_distance()
```

Get the drone ID

```
get_plane_id()
```

(The heading in the original reads "Get the drone's ToF height" — evidently a copy-paste slip, since the body describes the drone ID.)

Description: Gets the drone's ID.

Returns: integer — the drone ID.

Example:

```
ret = api.get_plane_id()
```

External electromagnet

```
Plane_cmd_electromagnet(type)
```

Description: Controls the add-on electromagnet.

Parameters:

- `type` — `2` = electromagnet grabs, `3` = electromagnet releases.

Example:

```
ret = api.Plane_cmd_electromagnet(2)
```

External clamp / electromagnet

```
Plane_cmd_clamp(type, angle = 0)
```

Description: Controls the add-on clamp and electromagnet.

Parameters:

- `type` — `0` disable the clamp, `1` enable the clamp, `2` set the clamp angle,

`3` electromagnet releases, `4` electromagnet grabs.

- `angle` — angle to rotate the clamp to, 0–180.

Example:

```
api.Plane_cmd_clamp(0)      # disable the clamp
api.Plane_cmd_clamp(1)      # enable the clamp
api.Plane_cmd_clamp(2,30)   # clamp must be enabled first; open the clamp to 30 degrees
api.Plane_cmd_clamp(3)      # electromagnet releases
api.Plane_cmd_clamp(4)      # electromagnet grabs
```